

TASK 1

1. 一张**RGB三通道彩色图片**是如何被处理成可用于训练的数据？**张量**又是什么？

RGB图片：3通道（第0轴） *height*（第一轴） *width*（第二轴）

处理：1.转换成tensor形式 2.归一化：从0 ~255颜色标号映射到0 ~1的区间 3.增加batch维度：把一批图片样本张量堆积成[batch,channel,height,width]

张量：多维数组（按维度分为第零轴、第一轴.....从外向内依次命名）

2. 一个卷积神经网络分为哪几层？

输入层→卷积层→激活层→池化层→卷积层→池化层→全连接层→输出层
（参照LeNet卷积神经网络）

3. 什么是卷积，数学严格意义上的卷积和神经网络中的卷积操作有区别吗？**卷积核**、**步长**是什么？

卷积：通过一个小模板在输入上滑动，用加权求和不断计算局部匹配程度

数学/CNN区别：1.函数移动+重叠计算（数学）/卷积核移动+局部加权求和（CNN） 2.数学的卷积需要上下、左右翻转，而CNN实际上没有翻转卷积核，只有平移操作-----因为反正数据都是模型自动学习出来的，没必要

卷积核：

可以理解成一个用来探测图像一小块局部信息的小窗口。一个卷积核用着一套权重（小窗格上的每个点用自己的一个权重）和一个偏置（小窗格上的每个点共用一个权偏置）。卷积核V在图X上移动，每次移动每个小窗格上的点采集其覆盖的像素点的特征（如明暗程度等等，通常一个卷积核处理一个特征），用自己的权重和共用的偏置计算得到与目标匹配程度H

步长：卷积核每次滑动元素的数量

4. **池化**有哪些类型，其作用分别有什么？**感受野**是什么？

池化类型：

- 最大池化（在池化窗口中保留最大值，保留区域的最强特征）
- 平均池化（取窗口所有数的平均值，保留区域整体信息）
- 全局最大池化（整个特征图只留下一个最大值数字）
- 全局平均池化（每个通道提取一个平均值）

作用：

- 保持平移不变性：针对图片偏移问题，降低卷积层对位置的敏感性
- 降低空间尺寸：例如把28×28的矩阵通过2×2池化窗口变为14×14，保留区域关键特征

感受野：在前向传播期间可能影响计算的所有元素（来自所有先前层），即计算出最终x的过程中所有初始值和中间值

5. **激活层**在此的作用？最后卷积神经网络会输出什么？

激活层作用：加入非线性函数，让模型有复杂的表达能力（如ReLU函数可以剔除经过卷积后不符合特征要求的负数元素）

最后会输出什么：得看目标（？）一般都是输出对各个分类的分数，后续还要经过softmax or 二分类函数变成概率（至少目前见到的卷积神经网络都是这样的）

TASK 2

假设当前文件结构：

```

data
|
├─ train
|   │
|   │   └─ rose
|   │       │
|   │       └─xxx.jpg
|   │           .....
|   │
|   └─ tulip
|       │
|       └─xxx.jpg
|           .....
|
└─ test
    │
    │   └─rose
    │       │
    │       └─xxx.jpg
    │           .....
    └─tulip
        │
        └─xxx.jpg
            .....

```

python源码:

```

import os
from PIL import Image
# 异常或者损坏的图片（假设图片在与.py程序并列的data文件夹里）
def check_image(path):
    bad_files=[]
    for root,dirs,files in os.walk(path):
        for file in files:
            if file.endswith('.jpg'):
                img_path=os.path.join(root,file)
                try:
                    img=Image.open(img_path)
                    img.verify()
                except:
                    bad_files.append(img_path)
    return bad_files
bad=check_image('./data')
print('损坏图片: ')
print(bad)

#划分数据集
if not bad==[]: #剔除坏文件
    for file in bad:
        os.remove(file)
from torchvision.datasets import ImageFolder
train_datasets=ImageFolder(root='./data/train') #ImageFolder负责：找到图片在哪里
test_datasets=ImageFolder(root='./data/test')
print(train_datasets.classes)
print(test_datasets.classes)

#数据增强&改变形状和张量（读取时输出这种）

```

```

from torchvision import transforms #transform负责：图片如何处理
train_transform=transforms.Compose([
    transforms.Resize((224,224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.ToTensor()
])
test_transform=transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor() #注意：这里的Totensor函数只是说之后读取时自动读取其张量形式，在
test_transform里储存还是路径+数字类别
])
train_datasets=ImageFolder(root='./data/train',transform=train_transform)
test_datasets=ImageFolder(root='./data/test',transform=train_transform)

#处理图片按批次喂给CNN
from torch.utils.data import DataLoader
train_loader=DataLoader( #注意：这里的train_loader本质上只是一个不断生成batch的迭代器，并没有真正储存
所有转化后的train_datasets
    train_datasets,
    batch_size=32,
    shuffle=True
)

```

效果：

```

(torch39-gpu) F:\ML_program\flower_juanji>D:\Miniconda\envs\torch39-gpu\python.exe f:/ML_program/flower_juanji/data/main.py
损坏图片：
[] (第一次运行的时候已经把坏图片删除了所以这里已经显示无损坏图片)
['daisy', 'dandelion', 'rose', 'sunflower', 'tulip']
['daisy', 'dandelion', 'rose', 'sunflower', 'tulip']

```

TASK 3

1. 什么是超参数？如何设定？

超参数：训练开始前人为设定的参数（学习率，批量大小，学习轮数等等）

如何设定：1.最开始列出多个大小的超参数，如 for lr in range([10, 1, 0.1 0.01])轮流学习并在验证集中比较效果 2.选取效果比较好的两个lr找到大致量级后通过观察验证集里的loss 和accuracy动态调整lr和epoch

2. 什么是优化器，有哪些类型？

优化器：更新模型参数，旨在让损失函数loss值降低的算法

类型：

- SGD（随机梯度下降）
- Momentum（动量梯度下降,不仅考虑当前梯度，还考虑之前梯度方向，使参数更新更加平）
- AdaGrad（自适应梯度）
- Adam（结合动量和自适应学习率）

3. DataLoader 与 DataSet是什么及其使用？（可调库）

- DataSet：图像分类保存器，将图像储存在（路径+数字类别标签），以便之后直接根据路径读取图像，根据数字类别标签读取图像类别

- DataLoader: 不断生成batch的迭代器, 当真正运行时抽取一定批量(batch)个图像 (可以规定是否随机), 将图像喂给CNN神经网络

注意: DataLoader真正到DataSet抽取前需要用transform库定义取出后的形式, 如transforms.Resize, transforms.ToTensor等等

- 使用:

* DataSet:

```
train_datasets=ImageFolder(root='./data/train')
```

* DataLoader:

```
train_loader=DataLoader(train_datasets,batch_size=32,shuffle=True)
```

4. 炼丹: 见源码 (在src文件夹里) 和结果 (在docs文件夹里)

(结果分为loss随轮次变化以及test_accuracy随轮次的变化, 其中一份是lr=0.001, epoch=100; 另一个是lr=0.01, epoch=200的结果-----目前还没有达到最好效果就停止, 是因为每次运行即使是gpu环境仍然训练耗时巨久, epoch继续增加就等不起了)

源码:

model.py部分:

```
model.py X
F:\ML_program > flower_identify > src > model.py
1 import torch
2 from torch import nn
3 import torch.nn.functional as F
4
5 class MyModel(nn.Module):
6     def __init__(self):
7         super(MyModel, self).__init__()
8         self.conv1 = nn.Conv2d(3, 96, kernel_size=(11, 11), padding=2, stride=4)
9         self.conv2 = nn.Conv2d(96, 256, kernel_size=5, padding=2, stride=1)
10        self.conv3 = nn.Conv2d(256, 384, kernel_size=3, padding=1, stride=1)
11        self.conv4 = nn.Conv2d(384, 384, kernel_size=3, padding=1, stride=1)
12        self.conv5 = nn.Conv2d(384, 256, kernel_size=3, padding=1, stride=1)
13
14        self.pool = nn.MaxPool2d(3, padding=0, stride=2)
15
16        self.fc1 = nn.Linear(6*6*256, 4096)
17        self.fc2 = nn.Linear(4096, 4096)
18        self.fc3 = nn.Linear(4096, 5)
19
20
21    def forward(self, x):
22        x=self.conv1(x)
23        x=F.relu(x)
24        x=self.pool(x)
25        x=self.conv2(x)
26        x=F.relu(x)
27        x=self.pool(x)
28        x=self.conv3(x)
29        x=F.relu(x)
30        x=self.conv4(x)
31        x=F.relu(x)
32        x=self.conv5(x)
33        x=F.relu(x)
34        x=self.pool(x)
35        x=torch.flatten(x, start_dim=1)
36        x=self.fc1(x)
37        x=F.relu(x)
38        x=self.fc2(x)
39        x=F.relu(x)
40        x=self.fc3(x)
41        return x
```

main.py部分:

```

1 import os
2 from PIL import Image
3 import torch
4 from torch import nn
5 from model import MyModel
6 import matplotlib.pyplot as plt
7 # 异常或者损坏的图片（假设图片在与.py程序并列的data文件夹里）
8 def check_image(path):
9     bad_files=[]
10    for root,dirs,files in os.walk(path):
11        for file in files:
12            if file.endswith((".jpg",".jpeg",".png",".bmp")):
13                img_path=os.path.join(root,file)
14                try:
15                    img=Image.open(img_path)
16                    img.verify()
17                except:
18                    bad_files.append(img_path)
19    return bad_files
20 bad=check_image('./data')
21 print('损坏图片: ')
22 print(bad)
23
24 #划分数据集
25 if not bad==[]: #剔除坏文件
26     for file in bad:
27         os.remove(file)
28 from torchvision.datasets import ImageFolder
29 train_datasets=ImageFolder(root='./data/train') #ImageFolder负责：找到图片在哪里
30 test_datasets=ImageFolder(root='./data/test')
31 print(train_datasets.classes)
32 print(test_datasets.classes)
33
34
35 #数据增强&改变形状和张量（读取时输出这种）
36 from torchvision import transforms #transform负责：图片如何处理
37 train_transform=transforms.Compose([
38     transforms.Resize((224,224)),
39     transforms.RandomHorizontalFlip(),
40     transforms.RandomRotation(20),
41     transforms.ToTensor()
42 ])
43 test_transform=transforms.Compose([
44     transforms.Resize((224,224)),
45     transforms.ToTensor() #注意：这里的ToTensor函数只是说之后读取时自动读取其张量形式，在test_transform里储存还是路径+数字类别
46 ])
47 train_datasets=ImageFolder(root='./data/train',transform=train_transform)
48 test_datasets=ImageFolder(root='./data/test',transform=test_transform)
49
50
51 #处理图片按批次喂给CNN
52 from torch.utils.data import DataLoader
53 train_loader=DataLoader( #注意：这里的train_loader本质上只是一个不断生成batch的迭代器，并没有真正储存所有转化后的train_datasets
54     train_datasets,
55     batch_size=32,
56     shuffle=True
57 )
58
59 test_loader=DataLoader(
60     test_datasets,
61     batch_size=32,
62     shuffle=False
63 )

```

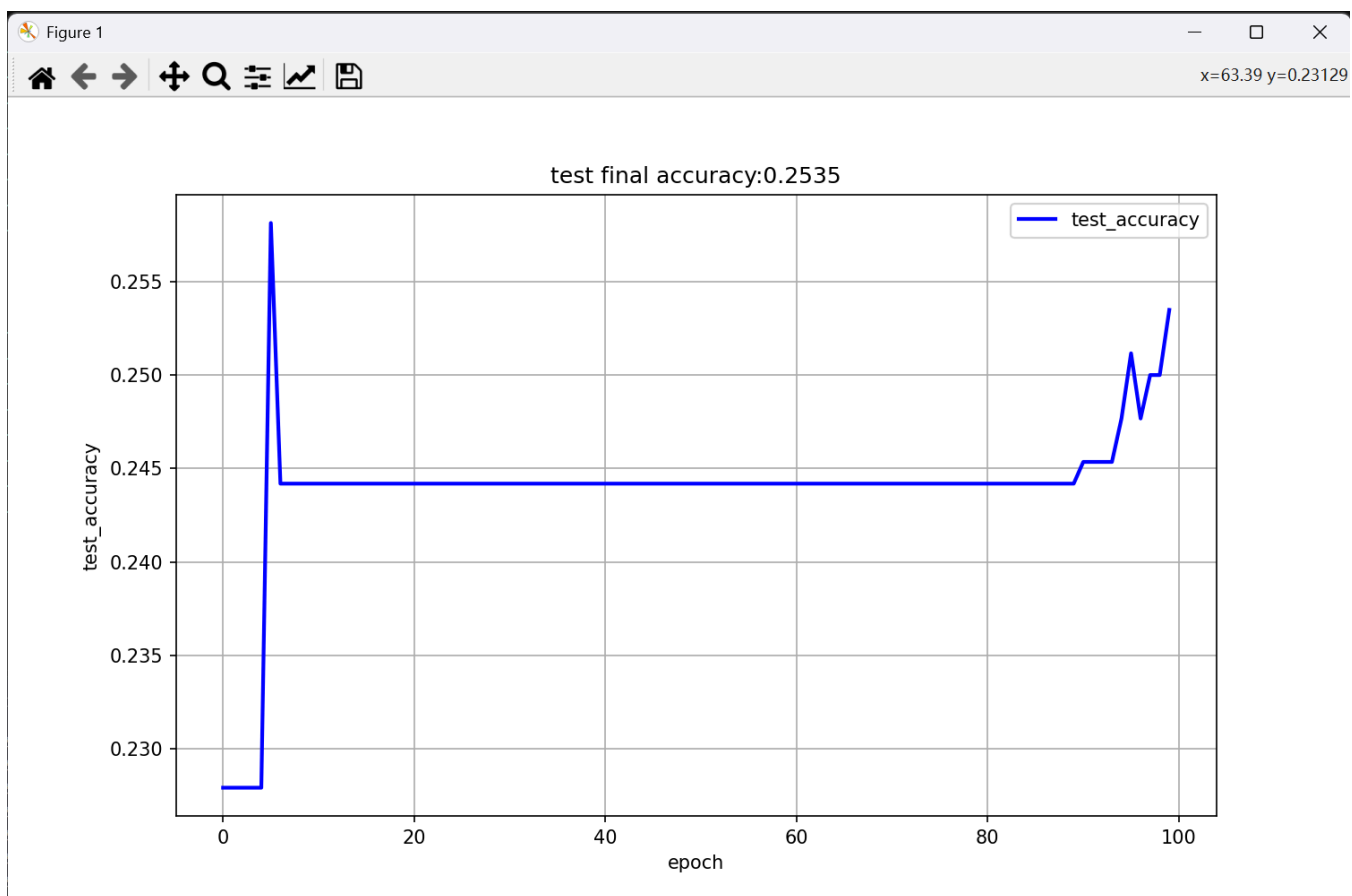
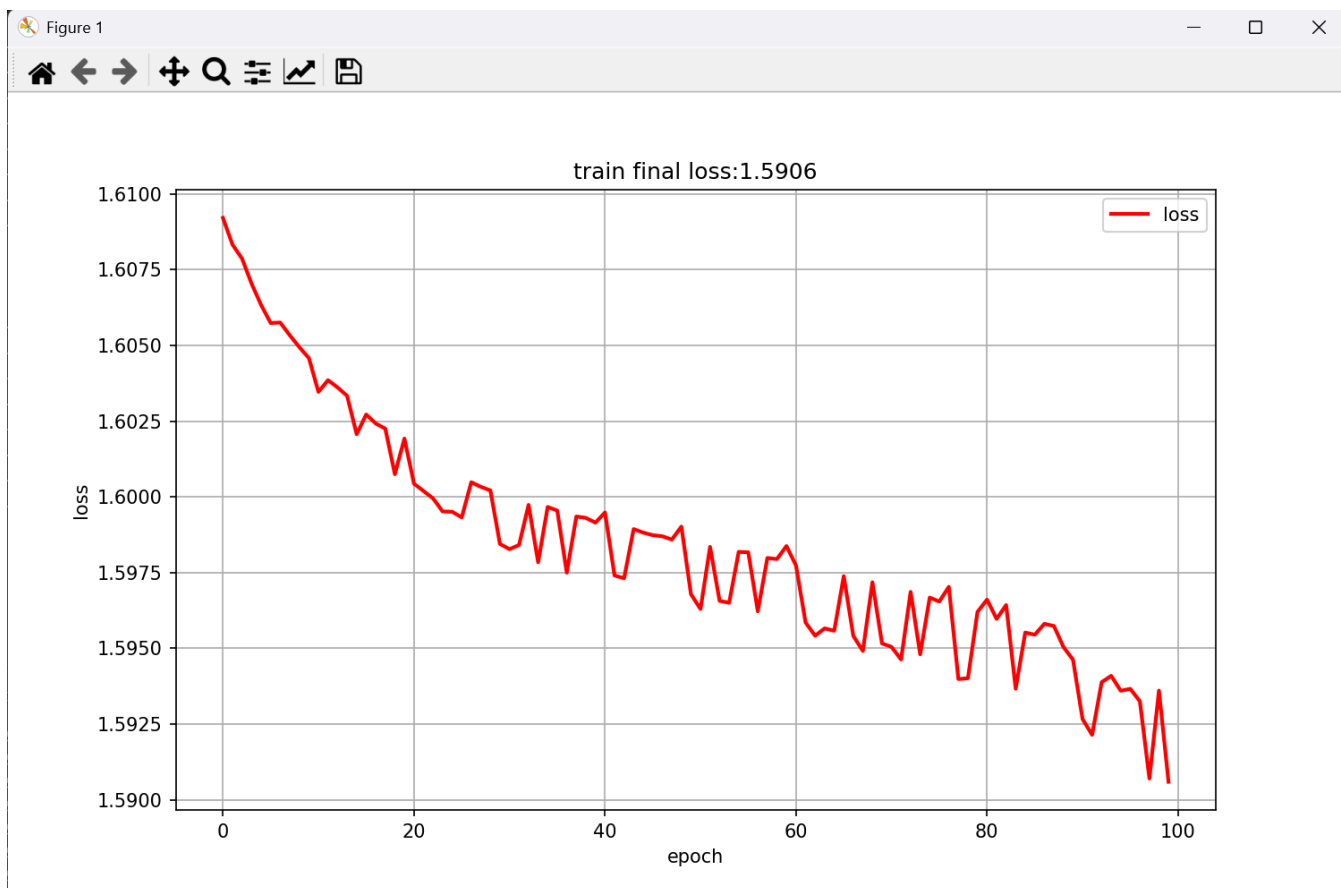
```

65 model = MyModel()
66 device="cuda"
67 model=model.to(device)
68 loss_fn = nn.CrossEntropyLoss()
69 optimizer=torch.optim.SGD(model.parameters(),lr=0.01)
70 loss_history=[]
71 epoch_history=[]
72 test_accuracy_history=[]
73 for epoch in range(100):
74     total_loss = 0
75     batch_num=0
76     for imgs,labels in train_loader:          #每次取出的imgs格式=torch.Size([32, 3, 224, 224]), torch不断迭代直到全部取出为止
77         batch_num += 1
78         imgs=imgs.to(device)
79         labels=labels.to(device)
80         outputs=model(imgs)
81         loss=loss_fn(outputs,labels)
82         optimizer.zero_grad()
83         loss.backward()
84         optimizer.step()
85         total_loss+=loss.item()
86     epoch_history.append(epoch)
87     loss_history.append(total_loss/batch_num)
88     with torch.no_grad():
89         correct=0
90         total=0
91         for imgs,labels in test_loader:
92             imgs=imgs.to(device)
93             labels=labels.to(device)
94             test_outputs=model(imgs)
95             values, indices = torch.max(test_outputs,1)
96             total+=labels.size(0)
97             correct+=(indices==labels).sum().item()
98     test_accuracy=correct/total
99     test_accuracy_history.append(test_accuracy)
100     if epoch%1==0:
101         print(f'当前训练轮次{epoch+1},loss={loss}')
102
103 torch.save(model.state_dict(),'flower_model1.1.pth')
104
105
106 plt.figure(figsize=(10, 6))
107 plt.plot(epoch_history,loss_history,color='red',linewidth=2,label='loss')
108 plt.title(f'train final loss:{loss_history[-1]:.4f}')
109 plt.xlabel('epoch')
110 plt.ylabel('loss')
111 plt.legend()
112 plt.grid(True)
113 plt.savefig('loss_curve1.1.png',dpi=300,)
114 plt.show()
115
116
117
118 plt.figure(figsize=(10, 6))
119 plt.plot(epoch_history,test_accuracy_history,color='blue',linewidth=2,label='test_accuracy')
120 plt.title(f'test final accuracy:{test_accuracy_history[-1]:.4f}')
121 plt.xlabel('epoch')
122 plt.ylabel('test_accuracy')
123 plt.legend()
124 plt.grid(True)
125 plt.savefig('accuracy_curve1.1.png',dpi=300,)
126 plt.show()

```

结果:

(1) 取lr=0.001, epoch=100



取lr=0.01, epoch=200:

